

```
1. # Python implementation - calculating average price
2. def calculate_average(prices):
3.     return sum(prices) / len(prices)
```

The Python version accomplishes the same task with significantly less code and greater clarity. This readability accelerates development and reduces errors—critical advantages when developing trading strategies.

Here's another simple example showing how Python can check if a stock price has gone up:

```
1. # Checking if today's price is higher than yesterday's
2. yesterday_price = 100
3. today_price = 105
4.
5. if today_price > yesterday_price:
6.     print("The stock went up!")
7. else:
8.     print("The stock went down or stayed the same.")
9.
```

Even if you've never coded before, you can probably understand what this Python code does!

?? Pro Tip: When writing trading algorithms, prioritize readability over premature optimization. Complex trading strategies can involve hundreds or thousands of lines of code. Clear, well-structured code makes debugging easier and helps you avoid costly trading errors. You can always optimize critical sections later if performance becomes an issue.

Extensive Libraries and Frameworks

Python's ecosystem includes thousands of specialized libraries that make financial analysis and trading more accessible. Key libraries include:

- **NumPy:** Provides high-performance mathematical functions and array operations
- **pandas:** Offers data structures and tools designed for time series and financial data analysis
-